

Software Engineer Coding Assessment

Candidate Handout — .NET Stack

This handout covers the full exercise. Your interviewer will introduce one or more additional requirements verbally during the session — that's expected and by design, not a sign you missed something in your first pass.

1. Overview & Format

This is a hands-on coding exercise used for Software Engineer hiring. You'll work from the core problem below, and your interviewer will introduce additional, previously-unseen requirements live so we can see how you extend a design you already built.

This runs one of two ways — your recruiter will tell you which applies, and will send a short separate note if there's anything to do before your session:

- Live, in one sitting: you build the full solution with your interviewer present throughout, then discuss it.
- Independent work first: you build the solution on your own beforehand, then meet your interviewer for a shorter live session covering the same requirements.

Either way, this is not a whiteboard exercise — you'll write and run real code.

2. Ground Rules

Tech stack — .NET

- Backend: ASP.NET Core Web API (C#), required. Minimal APIs or controller-based — your choice. Expose it as a small runnable API (REST is simplest) — your frontend needs something to call.
- Frontend: your choice of React or Angular, required. TypeScript is expected either way (Angular uses it by default; for React, TypeScript is preferred but plain JavaScript is fine). This is not optional — every session includes a working UI, regardless of role or level.
- Data layer: in-memory storage is completely fine for this exercise (a plain collection, or Entity Framework with an in-memory provider if you prefer) — you will not be scored on wiring up a real database.
- Visual polish isn't the point — a plain, unstyled page is completely fine. What matters is that the UI actually calls your API and renders real results.

Environment

- Use your own laptop and the IDE/editor of your choice.
- Make sure you can run your language's toolchain and test runner locally before the session (we'll send setup notes beforehand).

AI tools

- Use of AI coding assistants (e.g., Copilot, Claude Code, ChatGPT, Cursor) is allowed and encouraged — use them the way you would on the job.
- Be ready to explain, defend, and modify any AI-generated code yourself. If you can't explain why a piece of code is there, expect to be asked to change or remove it.
- The live requirement(s) your interviewer introduces are meant to be worked through with you narrating your reasoning — AI tools may help you write code, but the thinking needs to be visibly yours.

What we're evaluating

Broadly: correctness, code quality and testing, how you reason about trade-offs, and how you respond when a requirement changes. The exact bar we hold you to reflects the role you're interviewing for.

3. The Problem: Listing Search Service

The company ingests property listings from multiple data feeds (MLS providers). Your task is to build a small full-stack search experience over a set of listings — one working application, end to end.

Data model

Each listing is a JSON object with the fields below. A sample dataset (sample_listings.json, ~12 listings) is provided separately.

Field	Type	Notes
id	string	Unique per source (not necessarily unique across sources)
source	string	e.g. "MLS_A", "MLS_B" — the feed the listing came from
address	string	Free-text street address, not normalized
city / state / zip	string	May contain minor inconsistencies between sources
price	number	USD, list price
bedrooms	integer	
bathrooms	float	
sqft	integer	
latitude / longitude	float	
listedDate	string (ISO date)	Date the listing went live
status	string	"active" "pending" "sold"
description	string	Free text, searchable

What to build

A user should be able to search listings through a real UI, backed by real logic you implement yourself. Specifically, your application should support:

- Filtering by minPrice, maxPrice, minBedrooms, city, and a free-text keyword matched against the description.
- A targetBudget value, used along with recency to rank results by relevance — you design the scoring; we're more interested in whether your approach is sensible and clearly explained than in one "correct" formula.
- Pagination through results, with the user able to see and move between pages.
- Real inputs for all of the above, ranked results displayed for the current page (at minimum: address, price, bedrooms, and relevance score), and reasonable loading, no-results, and error states.

Handle invalid input deliberately rather than incidentally — for example, minPrice greater than maxPrice, a pageSize of zero or less, or a city with no matches. A clear error response beats a crash, a silent empty result, or wrong data.

Deliverables

- A working, runnable solution, wired together end to end.
- A test suite covering your core logic, including edge cases (no matches, tied scores, invalid filter values, pagination boundaries).
- A short README or comment block explaining your scoring approach and any trade-offs you made.

4. Questions?

Ask your interviewer if anything in the problem statement is ambiguous — for this exercise, asking good clarifying questions is itself a positive signal, not a sign you're behind.